

Week 6: Memory Management and Concurrency in Go and Rust

Aaron Foster

Southern New Hampshire University

CS 660: Advanced Programming Language Theory

Prof. Shadha Tabatabai

Aug 16th, 2026

Why These Two Languages

Go and Rust resolve the tension between memory safety and low-level control in opposite ways, which is what makes the pair worth comparing. Go is built for concurrent server workloads, with goroutines and channels as language primitives. Rust targets memory-sensitive contexts — embedded devices, browser engines, operating-system components — where collector pauses are unacceptable; Jung et al. (2021) call it the first industry-supported language to overcome the trade-off between high-level safety and low-level resource control. The pair isolates the variable of interest: whether safety is enforced at runtime or at compile time.

Runtime Memory Management

Go manages the heap with a concurrent, non-moving, tracing mark-sweep collector that works alongside the running application to reduce latency. GOGC sets the target heap as a multiple of the live heap; doubling it roughly halves GC CPU cost while doubling heap overhead (The Go Authors, n.d.). Rust has no runtime collector. Each value has one owner, and when that owner leaves scope the value is dropped and its memory returned to the allocator (Klabnik & Nichols, n.d.), so reclamation is deterministic and its cost visible in the source.

Resource Management Under Concurrency

Go's guidance is to share memory by communicating: goroutines pass values over channels, with mutexes and atomics for unavoidable shared state. This is ergonomic but not foolproof — Tu et al. (2019) found message passing responsible for a substantial share of blocking bugs in production Go, so the idiom promoted as safer introduced a deadlock class of its own. Rust binds resources to ownership: a value moved into a thread is unreachable from the

sender, shared state needs `Arc<Mutex<T>>`, and the lock guard releases on drop, so a forgotten unlock is not expressible.

Stack and Heap Allocation

Go's compiler places values by escape analysis: locals whose lifetimes it can determine go on the stack, while values it cannot bound escape to the heap and become the collector's responsibility (The Go Authors, n.d.). Goroutine stacks start small and grow on demand, making hundreds of thousands of goroutines practical. Rust puts compile-time-sized values on the stack and requires explicit indirection — `Box`, `Vec`, `String` — for heap data; the stack is last-in-first-out and fast, while the heap requires the allocator to find space and return a pointer (Klabnik & Nichols, n.d.).

Comparing Stack and Heap Behavior

Both resolve placement at compile time but reclaim differently. Go defers heap reclamation to a runtime collector, trading predictable timing for convenience; Rust resolves it statically, producing deterministic frees at the cost of a stricter type system. Go's collector can be tuned but never costs nothing, and its pauses are real, which matters for latency-sensitive work; Rust has no collector to tune, but every program must satisfy the borrow checker. Go's growable goroutine stacks also allow far greater concurrency density than Rust's fixed operating-system thread stacks.

Memory Models and Safe Concurrent Execution

The Go memory model defines happens-before over channels, mutexes, and atomics and guarantees data-race-free sequential consistency: a race-free program yields only outcomes explainable by some sequentially consistent interleaving of its goroutines (The Go Authors, 2022). Go does not prevent races, however. It makes their absence the programmer's obligation and supplies a runtime detector, leaving races a testing problem, and Tu et al. (2019) show real codebases fail that obligation regularly. Rust moves the obligation to compile time through Send and Sync: Send types may transfer ownership between threads, Sync types may be shared by reference, and violations are rejected at compile time (Klabnik & Nichols, n.d.). Safe Rust excludes data races by construction, though Jung et al. (2021) stress this covers the safe fragment; low-level libraries still need unsafe blocks verified separately.

Conclusion

The choice is where to pay for safety. Go offers faster development and high concurrency density but leaves data races to testing; Rust eliminates races and pauses by construction, at the cost of a steeper learning curve. Latency-sensitive or memory-constrained work favors Rust; throughput and delivery speed favor Go.

References

The Go Authors. (n.d.). *A guide to the Go garbage collector*. <https://go.dev/doc/gc-guide>

The Go Authors. (2022). *The Go memory model*. <https://go.dev/ref/mem>

Jung, R., Jourdan, J.-H., Krebbers, R., & Dreyer, D. (2021). Safe systems programming in Rust. *Communications of the ACM*, 64(4), 144–152. <https://doi.org/10.1145/3418295>

Klabnik, S., & Nichols, C. (n.d.). *The Rust programming language*.
<https://doc.rust-lang.org/book/>

Tu, T., Liu, X., Song, L., & Zhang, Y. (2019). Understanding real-world concurrency bugs in Go. *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 865–878.
<https://doi.org/10.1145/3297858.3304069>

Acknowledgment of AI Use

Per the course AI usage guidelines: Anthropic's Claude (Opus 5) was used to help edit this report (Anthropic, 2026). The author reviewed and approved the analysis and final recommendation.